

An Agentic AI Workflow to Simplify Parameter Estimation of Complex Differential Equation Systems

Saakaar Bhatnagar[†]

saakaar28@gmail.com

Sunnyvale, CA, USA

Keywords: Agentic AI; Parameter Estimation; Differentiable Physics; Ordinary Differential Equations (ODEs); JAX

Abstract

Parameter identification for mechanistic Ordinary Differential Equation (ODE) models underpins prediction and control in several applications, yet remains a manual and labor-intensive process: datasets are noisy and partial, models can be stiff and complex, and differentiable implementations demand framework expertise. An agentic AI workflow is presented that converts a lightweight, human-readable specification into a compiled, parallel, and differentiable model calibration pipeline. Users supply an XML description of the problem and fill in a Python code skeleton; the agent automatically validates consistency between problem definition and code, and auto-corrects pathologies in the input deck. It transforms Python callables into pure JAX functions for efficient just-in-time compilation and parallelization. The system then orchestrates a two-stage search comprising global exploration of the parameter space followed by gradient-based refinement. The result is an AD-native, reproducible workflow that lowers the barrier to advanced calibration while preserving expert control. An open-source implementation with a documented API and examples is released, enabling rapid movement from problem statement to interpretable ODE models with minimal effort.

1 Introduction

Ordinary differential equation (ODE) parameter fitting is a foundational task for turning mechanistic models or ideas into predictive tools across the sciences and engineering. In battery models, ODEs govern degradation kinetics, thermal behavior, and equivalent-circuit dynamics [1, 2]; accurate parameters are essential for state estimation, control, and safety analysis. In combustion modeling, reactions are often represented using Arrhenius ODE systems [3, 4]. In biology, systems of ODEs describe gene regulation, signaling, and epidemiological dynamics [5]. More broadly, ODE systems are frequently embedded within larger multiphysics simulations [2, 6] and digital twins, and poor calibration at the ODE level propagates downstream, degrading overall fidelity, stability, and decision quality.

However, in many instances, the parameters of the ODE models need to be estimated from data. Rigorous estimation of these parameters remains difficult in practice: real-world datasets are noisy and partial; models are often stiff or multiscale; and calibration requires repeated numerical solves which are computationally intensive and fragile to implementation details.

A variety of strategies exist to calibrate these ODE models to data. Problem-specific heuristic methods, where techniques such as equation linearization reduce the estimation problem to a linear regression [7–9], have been demonstrated in several use cases, but suffer from robustness and generalizability issues [10]. Population-based algorithms (like genetic algorithms or swarm optimization) have been successfully used in applications such as battery modeling [11, 12], structural engineering [13], and piezoelectrics [14]. However, population-based methods suffer from poor convergence properties near local minima, and scale poorly with search space dimension. This has been addressed in the past using

[†]Corresponding Author

gradient-based search methods near local optima. In particular, derivatives obtained using automatic differentiation instead of numerical differences have shown considerable success in solving ODE and PDE (Partial Differential Equation) model identification problems [15–17], particularly when models exhibit stiffness characteristics [10, 18]. By backpropagating through the ODE solve via algorithmic differentiation of the integrator or adjoint sensitivity methods, exact derivatives of the time-discretized simulation [19] output are obtained with respect to parameters, compared to numerical differencing which has inherent discretization errors.

However, practical adoption of efficient model identification algorithms is hindered by substantial software and workflow barriers. In particular, writing fully differentiable simulation code requires nontrivial fluency in state-of-the-art AD frameworks (e.g., PyTorch, JAX, Julia) and with their distinct programming models. For example, JAX’s functional, transformation-driven style (pure functions with explicit `jit/vmap/grad` semantics) can be unfamiliar for domain scientists. Beyond framework fluency, practitioners must hand-engineer complete optimization pipelines—defining objectives and constraints, organizing multi-experiment datasets, selecting solvers and tolerances, configuring sensitivity modes, and managing compilation and hardware particulars—each step demanding bespoke code and iterative tuning. These burdens reduce accessibility to modern capabilities such as reverse-mode automatic differentiation, just-in-time (jit) compilation, and advanced optimisers. This motivates tooling that abstracts these concerns and automates routine choices while preserving expert control.

Recent advances in artificial intelligence have delivered substantial improvements in user workflow efficiency, and several studies have investigated using LLMs to improve workflows in scientific computing. Kashefi and Mukerji [20] investigated the use of ChatGPT for writing and debugging numerical algorithms for a variety of complex problems. Dong et al. [21] and Pandey et al. [22] investigated using Large Language Models (LLMs) to automate the setup and running of complex fluid dynamics simulations, reducing the need for domain expertise and leading to smoother workflows. Further work has also explored using autonomous AI Agents to automate and accelerate engineering. Elrefaie et al. [23] propose a multi-agent framework for aerodynamic design, streamlining key steps in the automotive design process such as concept sketching, shape retrieval, meshing and simulation. Picard et al. [24] explored using Vision-Language Models (VLMs) in tasks relevant to engineering design such as similarity analysis, CAD generation, and topology optimization. These works demonstrated the potentially large impact AI agents and workflows could have in improving the efficiency of engineers and designers developing new products, similar to the already-realized productivity gains made in fields like software engineering and data analytics [25].

In this work, the use of agentic AI workflows is investigated to streamline parameter identification for complex ODE systems. The proposed framework abstracts away much of the human effort and multidisciplinary expertise traditionally required to formulate and solve these problems while remaining compatible with modern differentiable programming stacks. By delegating routine orchestration to an AI model and exposing only high-level controls to the user, the proposed workflow lowers the barrier to employing just-in-time compilation, automatic differentiation, and advanced optimization methods for ODE calibration. The result is a more accessible, reproducible, and efficient path from problem statement to calibrated model, without sacrificing numerical rigor or expert oversight. The framework is demonstrated on two common types of parameter estimation problems: recovering the parameters from simulated data, and obtaining models that best describe experimentally obtained data.

This work has the following features that would make it a valuable contribution:

1. **Simplified usage interface:** A human-readable XML interface and standard Python functions are used to define the problem. These are automatically translated into a well-posed constrained optimization in a few steps; capturing objectives, bounds, and constraints—thereby eliminating boilerplate scripting and reducing specification errors.
2. **AD-native, jit-compiled execution:** Standard Python callables are automatically transformed into pure JAX functions and staged for just-in-time compilation, enabling stable reverse-mode automatic differentiation and parallel execution (e.g., via `pmap`) without exposing users to framework-specific plumbing. This abstraction delivers differentiable, high-performance kernels from ordinary Python code.
3. **Automated validation of optimization setups:** The system performs automatic checks prior to fitting, identifying common pathologies such as infeasible or ill-conditioned inputs, misconfigured objective/constraint functions, and parameters declared but unused. The agentic setup

automatically corrects these issues, and informs users of further improvements they can make before launching the fitting problem. Early diagnostics eliminate incorrect parameter estimation runs, reducing cost.

4. **Open-source implementation:** The full codebase is released with a documented API and examples, enabling researchers and practitioners to adopt, audit, and extend the workflow for domain-specific applications. The code and experiments are available at the [GitHub link](#)

2 Proposed Workflow

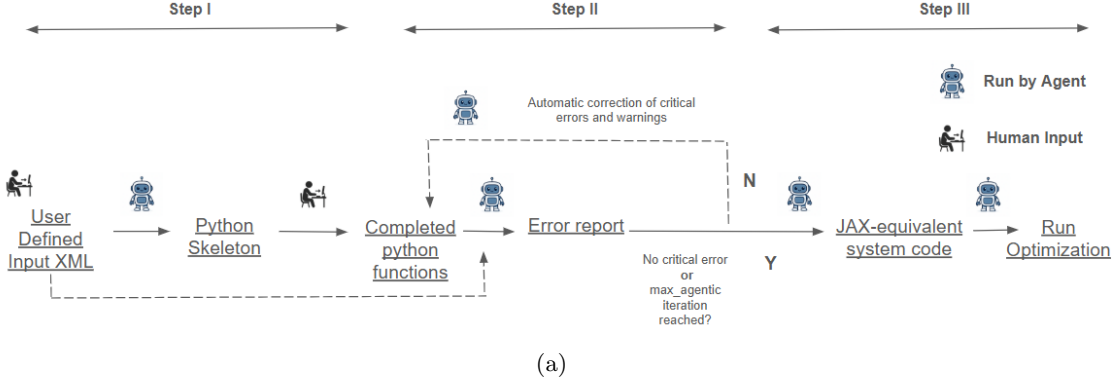


Figure 1: (a) The overall workflow proposed to automate ODE parameter estimation. Large parts of the workflow, which originally would have to be done by the user and would require expertise in programming, optimization and mathematics, can be handled by agentic AI.

A three-step, agentic workflow is presented that turns a lightweight, human-readable specification into a compiled, parallel, and differentiable parameter-estimation pipeline. The aim is to minimize manual setup: users describe the problem once, and the agent creates code skeletons, validates consistency, and executes a robust two-stage optimization—producing estimated parameters and solutions requested by the user. The workflow can be visualized in Figure 1, and is briefly described:

1. **Step I:** The user supplies a human-readable XML spec (states, parameters/bounds, dataset). The agent creates a minimal Python skeleton (ODE, loss, quantities of interest) to fill out, so only domain logic must be filled in.
2. **Step II:** After the user completes the functions, the agent cross-checks XML and code, validates shapes/signatures/bounds, separates critical errors from warnings, and auto-remediates issues iteratively.
3. **Step III:** The agent converts Python callables to JAX code, `jit`-compiles pure functions, sets parallelization, runs a two-stage optimization (PSO for global search, then gradient refinement), and writes fitted parameters and reports to file.

2.1 Step I: Code Skeleton Generation

The user provides a human-readable XML file that defines the parameter-estimation task (state variables, parameters and bounds, and a dataset). The AI model ingests this specification and emits a minimal Python code skeleton containing function definitions of the ODE system, the loss functional over simulated trajectories, and any quantities of interest to be reported—so that users implement only domain logic. The parts of the function to be filled in by the user are clearly highlighted. Figure 2 demonstrates a sample input-output to the AI model in this step. The user provides an input XML (which is filled using a template) to the model, and the model provides a verbose code skeleton designed to minimize user effort to populate.

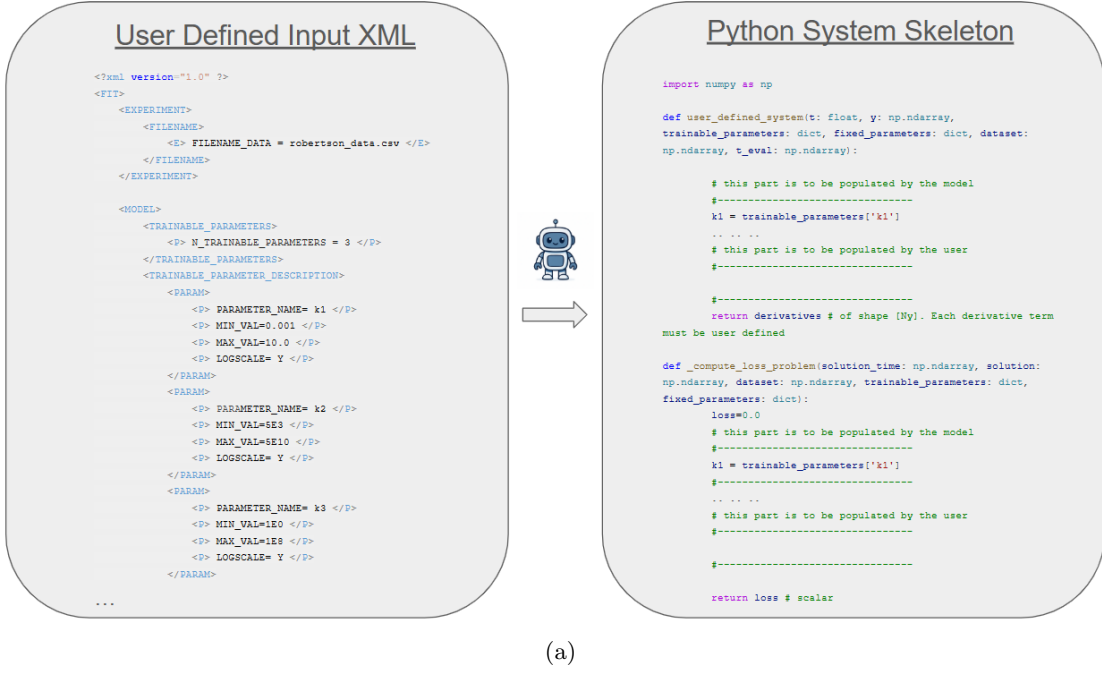


Figure 2: (a) Step I of the workflow: Conversion of the input XML to a Python function skeleton. This skeleton contains clearly demarcated sections to be filled in by the user.

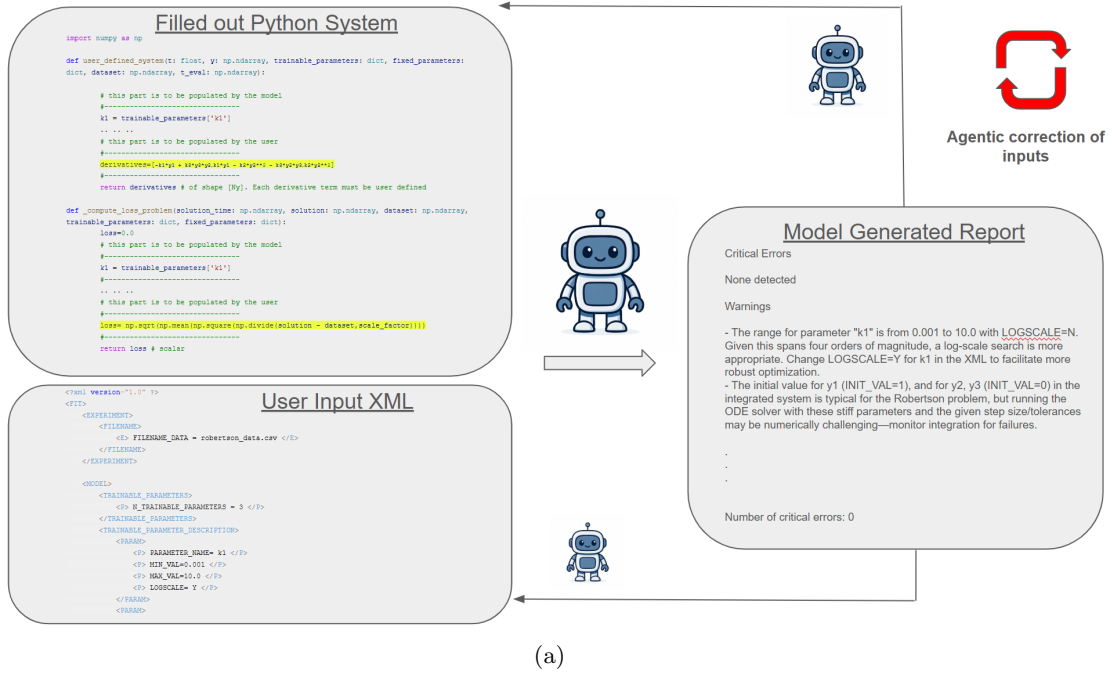


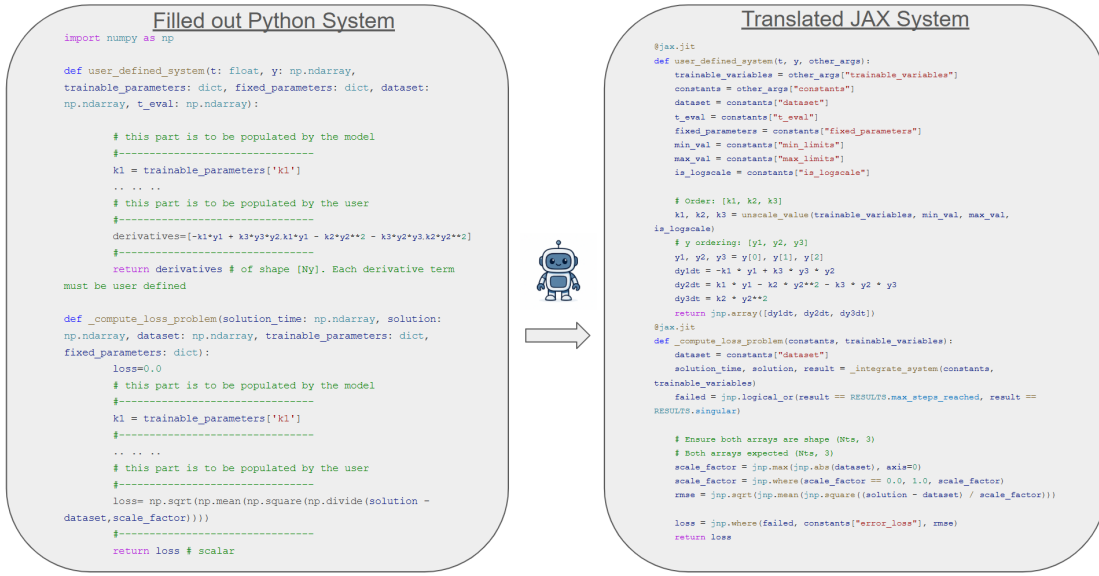
Figure 3: (a) Step II takes the completed (completions highlighted) Python functions and input XML, and generates a report containing critical errors and warnings. The agent automatically corrects the input files based on issues observed, and then reassesses the inputs. This is done in a loop until all critical errors are fixed, or a maximum number of iterations is reached.

2.2 Step II: Setup Error Corrections

After the user completes the Python functions, the XML and Python callables are resubmitted to the agent for cross-checking. The agent performs input validation, checking consistency between XML

declarations and code (for example, checking for correct function signatures, correct array shapes within functions, and any unused parameters), detection of implementation pitfalls, and diagnosis of potentially suboptimal optimization choices (e.g., ill-scaled variables, infeasible bounds). Findings are partitioned into critical errors—conditions likely to cause solver failure—and warnings—issues that may degrade accuracy or efficiency. Based on this report, the agent automatically attempts to fix some of the identified faults, while broadcasting changes made to the user. This is done in a loop until the maximum number of agentic iterations is reached, or no more critical errors are detected. Figure 3 describes the workflow in this step; the agent reads the input XML and completed functions, and examines the setup for errors and inconsistencies. It auto-iterates to correct any critical errors, and apprises the user of more minor issues. Appendix A.3 demonstrates examples of the workflow, showing how the agent corrects user inputs.

2.3 Step III: Compilation, Optimization and Post Processing



(a)

Figure 4: (a) The corrected system is translated into pure JAX functions that are jit-compilable, fully differentiable and parallelized using primitives like pmap. The functions are then used to solve a two-stage optimization problem to do parameter estimation.

Once critical issues are resolved (and warnings acknowledged), the agent transforms the user code into JAX-compatible, pure functions and applies jit compilation and parallelization primitives (e.g., pmap) as appropriate. This is shown in Figure 4. The resulting pipeline executes a two-stage optimization:

1. Global Search: A globally explorative particle-swarm optimization (referred to as PSO henceforth) to locate promising regions of the parameter space, parallelized across devices (See Appendix A.1 for details).
2. Local Refinement: A gradient-based refinement stage, where derivatives are obtained via automatic differentiation through the ODE solve, enabling stable, high-precision updates. This is done similarly to Neural ODEs, and is referred to as the NODE stage henceforth (See Appendix A.2 for details).

The two-stage process is commonly used in optimization, and this overall arrangement preserves ease of use while delivering the performance and reproducibility benefits of differentiable, jit-compiled, parallelized simulation. Once the optimization is complete, the resulting parameters and other information requested by the user are written to output files.

3 Results

3.1 Estimating parameters from simulated data

This section demonstrates parameter estimation results on synthetic datasets generated from known ODE models. Using simulated data is valuable when the practitioner has high confidence that the mechanistic model faithfully represents the data-generating process, or when parameters must be re-estimated from new observations after changes in operating conditions, aging, or maintenance. They serve as a valuable first test of estimation workflows before using them on experimental data.

3.1.1 Robertson’s equation

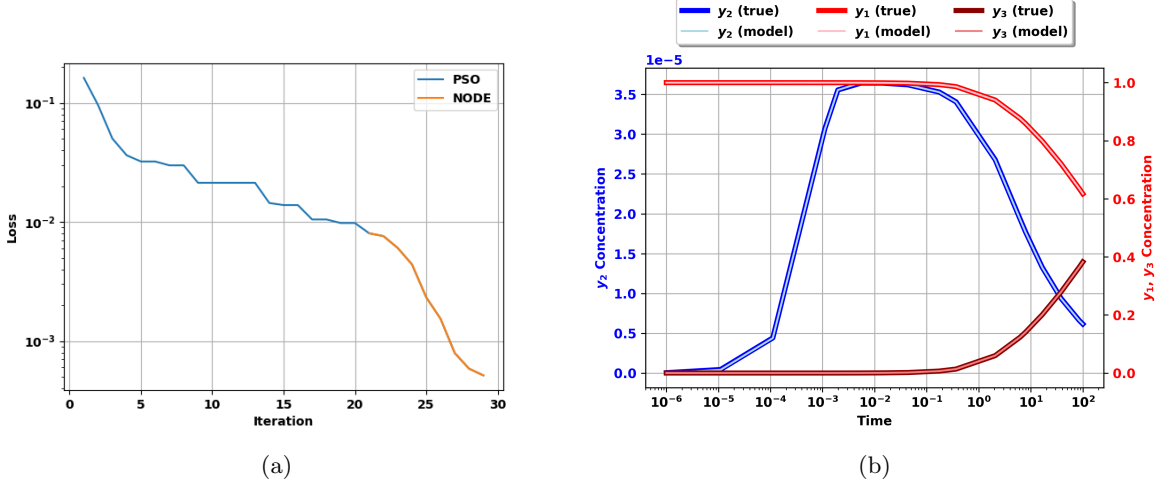


Figure 5: (a) Fit loss showing the 2-stage loss evolution (b) Fitting result showing the accuracy of the obtained fit for solution variables of all time scales

Robertson’s kinetics [26] provide a classical and demanding benchmark for ODE modeling pipelines. The system is markedly stiff—reaction rates span several orders of magnitude—so trajectories feature rapid transients followed by long quasi-stationary regimes. These multiscale test solver stability and error control, making Robertson’s problem a stringent test for parameter identification algorithms [27] and for surrogate modeling approaches [28].

The equation system, with rate parameters (k_1, k_2, k_3) is given as:

$$\dot{y}_1 = -k_1 y_1 + k_2 y_2 y_3, \quad (1)$$

$$\dot{y}_2 = k_1 y_1 - k_2 y_2 y_3 - k_3 y_2^2, \quad (2)$$

$$\dot{y}_3 = k_3 y_2^2, \quad (3)$$

and the classical parameter values are $k_1 = 0.04$, $k_2 = 3 \times 10^7$, $k_3 = 10^4$. The system state is described by the vector $[y_1, y_2, y_3]$. The aim of this experiment is to efficiently identify these parameters, starting with a continuous hypercube of possible values.

In this work, the loss is computed as

$$\text{MSE} = \frac{1}{N} \sum_{k=1}^N \frac{1}{3} \|\hat{\mathbf{y}}(t_k) - \mathbf{y}(t_k)\|_2^2.$$

Figure 5a shows the iterative fitting algorithm improving the fit and identifying the parameters of the problem. The PSO stage performs a global search over the parameter space, and the gradient-based search fine-tunes the obtained solution. Figure 5b shows the quality of the obtained fit, with the obtained parameters predicting the solution well. Table 1 shows the estimated versus original values; the results show good agreement. The experimental results, including the results of each workflow step, are available at the [GitHub link](#)

Parameter Name	True Parameter	Predicted Parameter
k_1	4.0e-02	4.010e-02
k_2	3.0e+07	3.009e+07
k_3	1.0e+04	1.005e+04

Table 1: True v/s predicted parameters for the Robertson system

3.1.2 Van der Pol Oscillator

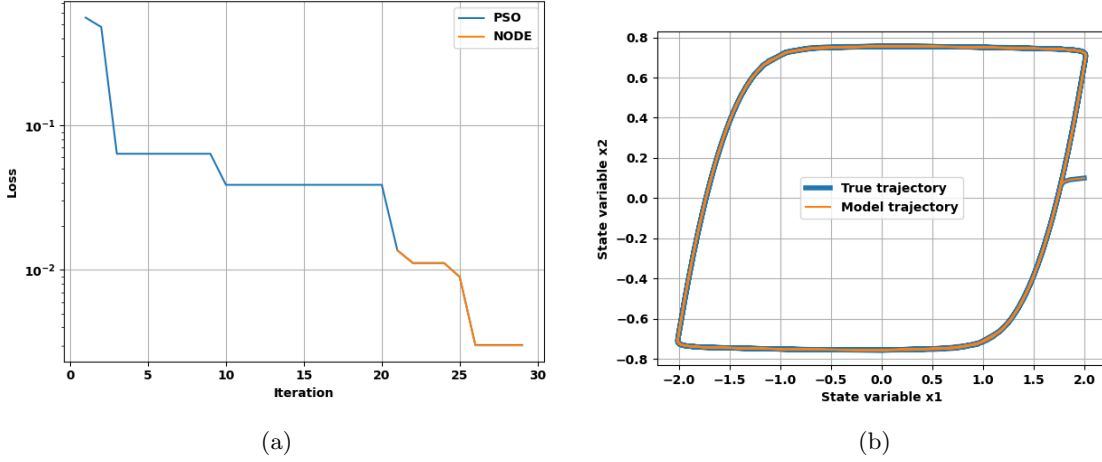


Figure 6: (a) Fit loss showing the 2-stage loss evolution (b) Fitting result showing the accuracy of the obtained fit

The Van der Pol oscillator is a canonical relaxation oscillator that appears across electronics, biomechanics, and mechanical vibrations [29, 30]. Solutions of the system exhibit long intervals of slow drift punctuated by rapid transitions, producing a characteristic non-sinusoidal limit cycle. These multiscale behaviors make the model a compact but stringent test for calibration pipelines, stressing both numerical integration (stiffness at large nonlinearity levels) and optimizer robustness.

In this work, the scaled Liénard-plane formulation is used in our demonstration. Let x_1 denote the primary state (e.g., displacement or voltage) and x_2 an auxiliary state (e.g., scaled velocity or charge). The parameter $\mu > 0$ controls the degree of nonlinearity and the separation of time scales:

$$\dot{x}_1(t) = \mu \left(x_2(t) - \left(\frac{1}{3} x_1(t)^3 - x_1(t) \right) \right), \quad (4)$$

$$\dot{x}_2(t) = -\frac{1}{\mu} x_1(t). \quad (5)$$

For small μ , trajectories are nearly harmonic; for large μ , the system develops pronounced relaxation oscillations with sharp fast segments. In this experiment, the true value of $\mu = 10$ is estimated from observed time series data.

The loss function is computed as

$$\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N \|\hat{\mathbf{x}}(t_k) - \mathbf{x}(t_k)\|_2^2, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

Figure 6a shows the loss evaluation through iterations. The gradient-based fine tuning improves the quality of the fit, and Figure 6b shows the fit, which is in excellent agreement with data. Table 2 shows the obtained μ_{est} from the estimation process; it shows excellent agreement with the value of μ_{sim} that generated the simulation data. The experimental results, including the results of each workflow step, are available at the [GitHub link](#)

Parameter Name	True Parameter	Predicted Parameter
μ	1.0e+01	1.0005e+01

Table 2: True v/s predicted parameter for the Van der Pol system

3.2 Estimating parameters from experimental data

This section explores using experimental data to estimate ODE models. This represents a more difficult class of problems, as the data may be sparse or not representative to accurately capture all phenomena of interest, resulting in an ill-posed estimation problem. In addition, experimental data is often noisy, unevenly sampled, and is not available for all integrable variables in the ODE model. Further, the ODE models being estimated are only approximate representations of the physics, making perfect matches to experiment unlikely. The ODE systems are usually inspired by physically complex, already understood equations, and in many cases are difficult to estimate using experimental data. In subsequent subsections, the framework is demonstrated on such examples inspired by real-world, practically relevant studies.

3.2.1 Piezoelectric Actuator Modeling

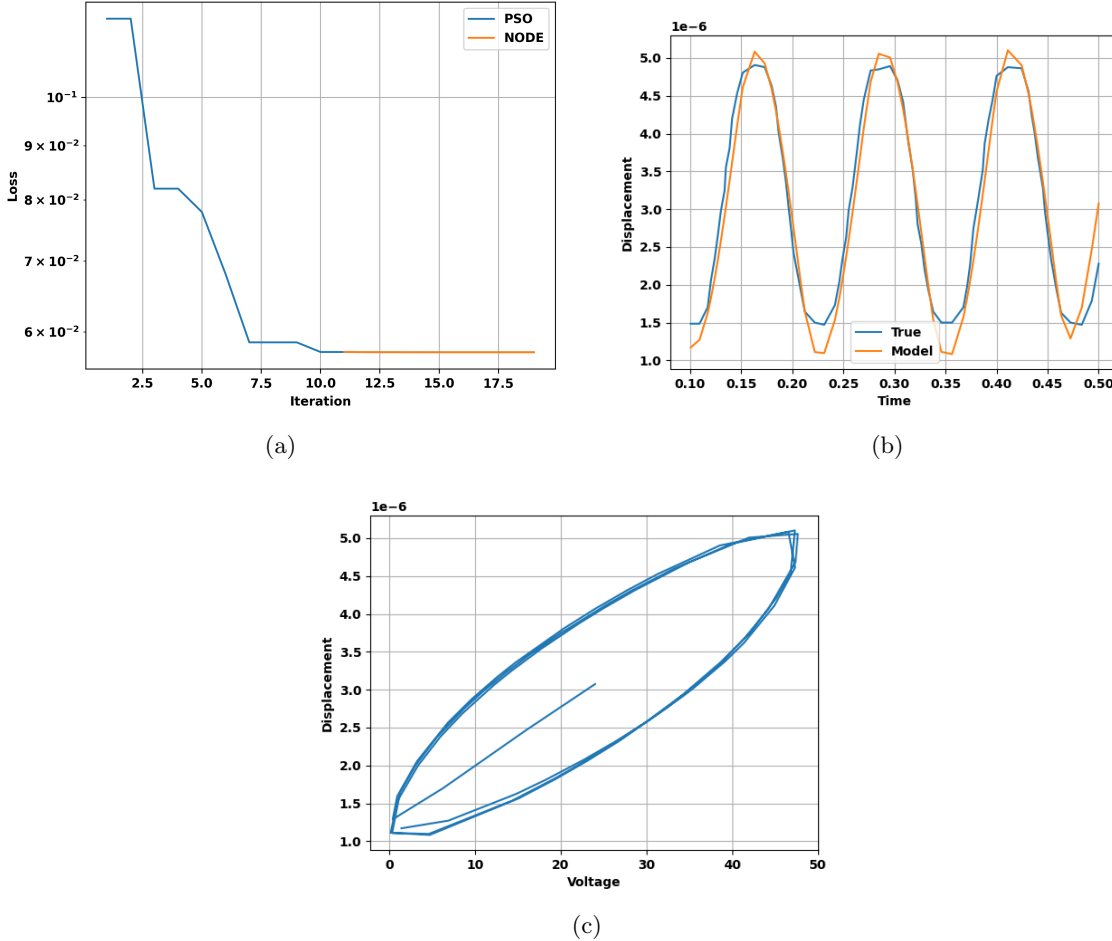


Figure 7: (a) Loss function evolution. The gradient-based optimization only minimally improves the fit (b) Predicted v/s experimental displacement (c) Expected hysteresis loop from parameters.

The framework is demonstrated on a piezoelectric actuator experiment (inspired from the study by Ha et al. [14]) exhibiting rate-dependent hysteresis, modeled via the Bouc–Wen formulation and

its voltage-driven variant. This benchmark couples nonlinear hysteresis with second-order mechanical dynamics, producing characteristic loops and transient behavior that the model must capture. The goal is to estimate the hysteresis and plant parameters from measured displacement under a known voltage drive.

The classical Bouc–Wen hysteresis is posed for a state z driven by a kinematic input x :

$$\dot{z} = \alpha \dot{x} - \beta |\dot{x}| z |z|^{n-1} - \gamma \dot{x} |z|^n, \quad (6)$$

with shape parameters α, β, γ and exponent n . For piezoelectric actuation, an internal hysteresis state h is driven directly by the voltage $V(t)$:

$$\dot{h} = \alpha d_e \dot{V} - \beta |\dot{V}| h - \gamma \dot{V} |h|, \quad (7)$$

where d_e is the piezoelectric coefficient. In this experiment the applied voltage is

$$V(t) = 24 + 24 \sin(16\pi t). \quad (8)$$

The actuator mechanics are modeled as a single degree-of-freedom mass–spring–damper system with a hysteresis-corrected drive:

$$m_p \ddot{x}_p + c_p \dot{x}_p + k_p x_p = k_p (d_e V - h), \quad (9)$$

where $x_p(t)$ is the measured displacement, and m_p, c_p, k_p denote the effective mass, damping, and stiffness. The parameter vector to be estimated is $\theta = (\alpha, \beta, \gamma, c_p, k_p, d_e)$, with initial conditions $[x_p(0), \dot{x}_p(0), h(0)] = [0, 0, 0]$ chosen consistently with the experiment. In this work, n is set to 1, and m_p to 0.1, consistent with the used reference. The goal is to match the model to experimental displacement data, ignoring the initial transients.

Let N be the number of data samples, with experimental displacement d_i^{exp} , simulated displacement d_i^{sim} , and scale factor s . The loss is

$$\mathcal{L} = \sqrt{\frac{1}{N} \sum_{i=1}^N \left(\frac{d_i^{\text{exp}} - d_i^{\text{sim}}}{s} \right)^2}. \quad (10)$$

The scale factor is set as the maximum value of displacement in the dataset. Figure 7a shows the loss function evolution with iteration. It is notable that the improvement by the gradient-based iteration steps improve the solution minimally; this could indicate that the PSO optimizer finds a solution that is good enough. This is confirmed by examining Figure 7b, showing a comparison of x_p with experimental data; the proposed parameters return a good match. Figure 7c shows the expected hysteresis relationship between x_p and applied voltage, further confirming the validity of the obtained model. Table 3 shows the parameters obtained; in the present estimate, the model sets the value of β to zero. Although it returns a good estimate of the experimental data, this can be avoided by setting the search range of β to nonzero values in the input XML. The experimental results, including the results of each workflow step, are available at the [GitHub link](#)

Parameter Name	Predicted Parameter
α	5.5947e-01
$\beta(V^{-1})$	0.0000e+00
$\gamma(V^{-1})$	2.2605e-09
$c_p(kg/s)$	2.5651e+01
$k_p(kg/s^2)$	1.0800e+07
$d_e(m/V)$	1.2793e-07

Table 3: Estimated parameters for the piezoelectric actuator model

3.2.2 Battery Thermal Runaway

In this section, Arrhenius ODE models are fit to experimental Accelerating Rate Calorimetry (ARC) data. These models play a critical role in modeling thermal failure in battery packs due to overheating, mechanical damage or age. The ARC data in this experiment is taken from the data made available in the study by Schöberl et al. [31], and details on how ARC experimental data is collected can be found in the report by Giuliano et al. [32].

The system of equations that are commonly used to fit thermal runaway calorimetry data are Arrhenius reaction kinetic equations [2, 7, 10]. In this experiment, the model takes a 2-equation form

$$\dot{c}_1(t) = -A_1 \exp\left(-\frac{E_{a1}}{k_b T(t)}\right) c_1(t), \quad (11)$$

$$\dot{c}_2(t) = A_2 \exp\left(-\frac{E_{a2}}{k_b T(t)}\right) c_2(t)^{n_2} (1 - c_2(t))^{m_2}, \quad (12)$$

$$\dot{T}(t) = |h_1 \dot{c}_1(t)| + |h_2 \dot{c}_2(t)|, \quad (13)$$

where c_1 and c_2 are the normalized reactant concentrations, A_1 and A_2 are frequency factors, E_{a1} and E_{a2} are activation energies, h_1 and h_2 are normalized reaction enthalpies, m_2 and n_2 are reaction exponents, and T is temperature. The loss is computed as shown in Equation 14:

$$Loss = \lambda_1 \sum_t \left(\log_{10} \left(\frac{dT}{dt} \right)_{data} - \log_{10} \left(\frac{dT}{dt} \right)_{predicted} \right)^2 + \lambda_2 \sum_t (T_{data} - T_{predicted})^2, \quad (14)$$

Parameter Name	Predicted Parameter
$A_1(s^{-1})$	4.057e+11
$E_{a1}(J)$	2.000e-19
$h_1(K)$	2.519e+02
$A_2(s^{-1})$	1.220e+17
$E_{a2}(J)$	2.565e-19
$h_2(K)$	4.970e+01
m_2	4.744
n_2	2.407

Table 4: Estimated parameters for the battery thermal runaway model

The loss function captures the difference in predicted temperature, as well as temperature rate, with λ_1 and λ_2 weighting the relative loss terms.

Figure 8a shows the loss function with iteration; the gradient-based method improves the obtained solution significantly after PSO is complete. Figure 8b compares the temperature reading versus time; the obtained model captures important trends such as slow self heating and rapid thermal runaway. Figure 8c compares the predicted temperature rate to experiment; the model captures the salient schemes of the thermal runaway process, including the slow self-heating and thermal runaway onset. There exist more advanced algorithms [2] to obtain such models from ARC data, however they require significant domain expertise to set up. The current approach enables the solution of the estimation problem with little effort, using AI agentic workflows, and users can use the obtained models as a starting point for their modeling problems. The experimental results are available at the  [GitHub link](#)

4 Discussion and Conclusion

This work’s primary contribution is an agentic AI workflow that turns high-level problem intent into an executable, performant parameter-estimation pipeline with minimal user effort. Given human-readable

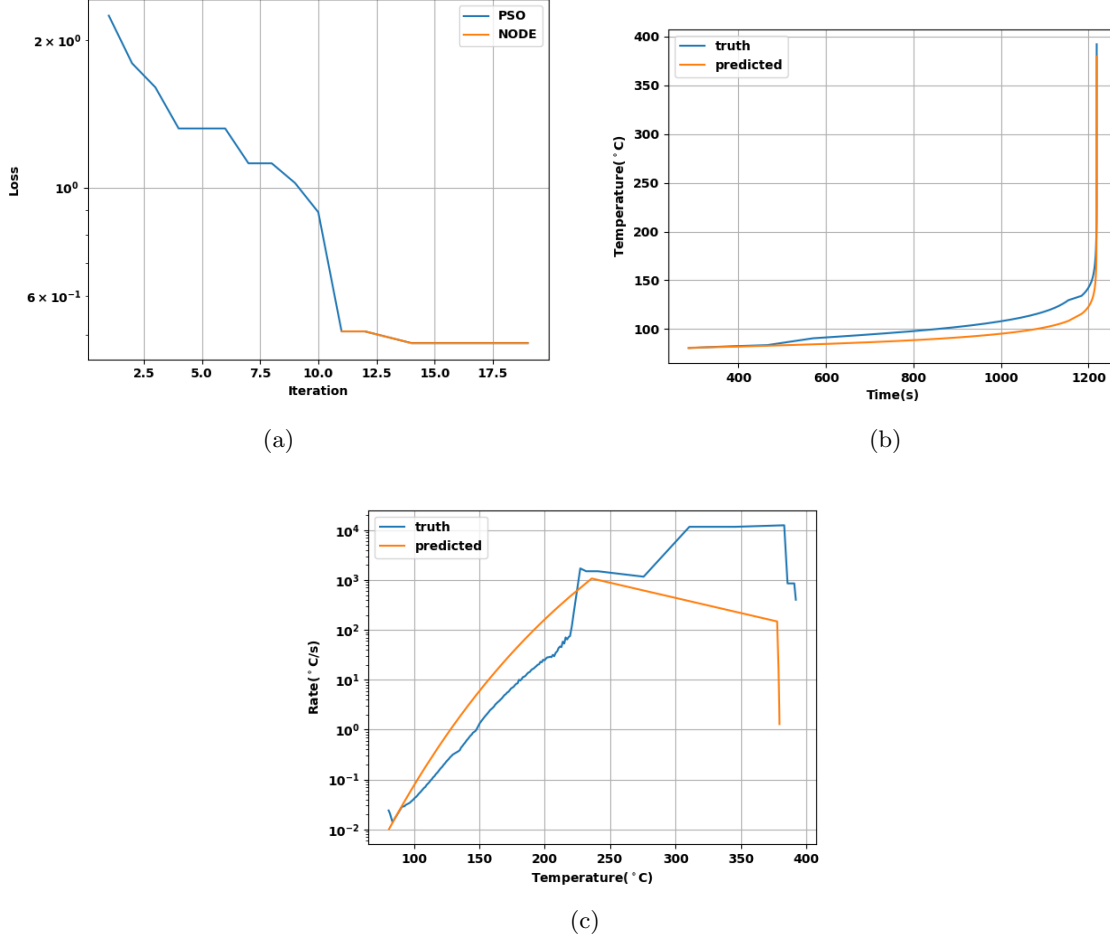


Figure 8: (a) Loss function evolution during parameter estimation (b) Model match to experimental temperature data (c) Model match to experimental temperature rate

user inputs, the agent automatically creates Python routine skeletons for users to fill, translates user routines into `jit`-able kernels, checks user inputs for correctness, and orchestrates a global-then-local optimization. The framework is demonstrated on data obtained from simulations of stiff ODE systems, showing that the generated setup is able to recover the parameters that setup the simulation successfully. The framework was also demonstrated on experimental data from several real-world experiments, and the framework returned approximate ODE model systems that are good representations of the experimental data.

A few other pertinent observations were made while testing the proposed framework:

1. Model-data mismatch: Mechanistic models are approximations; measurement noise, unmodeled effects and imperfect models reduce the ability of the models to approximate the data.
2. Numerical sensitivity of gradient computation: Reverse-mode/adjoint gradients are sensitive to integration tolerances and step-size control. The agent treats tolerances as critical hyperparameters, and is usually needed to be tuned by the user. Rackauckas [33] demonstrated that forward-integral stability does not guarantee reverse-mode derivative calculation stability, and the tolerances on the integration may need to be tighter if the user wishes to backpropagate through the solution.
3. The agentic workflow is seen to be highly effective in correcting user errors and suboptimal user choices. However, some hyper-parameters like ODE solver settings and time-stepping methods still need to be tuned for effective parameter estimation.

The net contribution of the tool is a dramatic reduction in setup friction: users get to a baseline fit quickly, then iterate on modeling choices instead of configuring optimization problems, dealing with numerical details, differentiability and parallelization infrastructure.

This work is intended to be a foundation in intelligent inverse problem solutions, and can be extended in several ways. PSO can be augmented or replaced with Bayesian optimization to cut simulation budgets and improve global search in stiff, weakly identifiable spaces. The work can be extended to include PDE parameter estimation by including new estimation methods like Physics Informed Neural Networks (PINNs) [34] and setting up differentiable finite element or finite volume simulations using frameworks such as PhiFlow [35]. Further extension of the agent can be done to automate hyper-parameter tuning and ODE solver settings; this includes solver tolerances, time-stepping methods and stepsize controller settings.

5 Funding Sources

This research received no specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author(s) used GPT in order to check grammar and format language for better flow. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

References

- [1] Manh-Kien Tran, Andre DaCosta, Anosh Mevawalla, Satyam Panchal, and Michael Fowler. Comparative study of equivalent circuit models performance in four common lithium-ion batteries: Lfp, nmc, lmo, nca. *Batteries*, 7(3):51, 2021.
- [2] Saakaar Bhatnagar, Andrew Comerford, Zelu Xu, Simone Reitano, Luigi Scrimieri, Luca Giuliano, and Araz Banaeizadeh. A layered swarm optimization method for fitting battery thermal runaway models to accelerating rate calorimetry data. *Journal of The Electrochemical Society*, 2024.
- [3] Marco Mehl, William J Pitz, Charles K Westbrook, and Henry J Curran. Kinetic modeling of gasoline surrogate components and mixtures under engine conditions. *Proceedings of the Combustion Institute*, 33(1):193–200, 2011.
- [4] Jan G Verwer. Gauss–seidel iteration for stiff odes from chemical kinetics. *SIAM Journal on Scientific Computing*, 15(5):1243–1250, 1994.
- [5] Stefano Giampiccolo, Federico Reali, Anna Fochesato, Giovanni Iacca, and Luca Marchetti. Robust parameter estimation and identifiability analysis with hybrid neural ordinary differential equations in computational biology. *NPJ Systems Biology and Applications*, 10(1):139, 2024.
- [6] B Michaelis and B Rogg. Fem-simulation of laminar flame propagation. i: Two-dimensional flames. *Journal of computational physics*, 196(2):417–447, 2004.
- [7] Paul T Coman, Eric C Darcy, Christian T Veje, and Ralph E White. Modelling li-ion cell thermal runaway triggered by an internal short circuit device using an efficiency factor and arrhenius formulations. *Journal of The Electrochemical Society*, 164(4):A587, 2017.
- [8] Yu Wang, Dongsheng Ren, Xuning Feng, Li Wang, and Minggao Ouyang. Thermal kinetics comparison of delithiated li [nixcoymn1-xy] o2 cathodes. *Journal of Power Sources*, 514:230582, 2021.
- [9] Dongsheng Ren, Xiang Liu, Xuning Feng, Languang Lu, Minggao Ouyang, Jianqiu Li, and Xiangming He. Model-based thermal runaway prediction of lithium-ion batteries from kinetics analysis of cell components. *Applied energy*, 228:633–644, 2018.

- [10] Saakaar Bhatnagar, Andrew Comerford, Zelu Xu, Davide Berti Polato, Araz Banaeizadeh, and Alessandro Ferraris. Chemical reaction neural networks for fitting accelerating rate calorimetry data. *Journal of Power Sources*, 628:235834, 2025.
- [11] Yu-Shan Cheng. Identification of parameters for equivalent circuit model of li-ion battery cell with population based optimization algorithms. *Ain Shams Engineering Journal*, 15(3):102481, 2024.
- [12] Md Ashiqur Rahman, Sohel Anwar, and Afshin Izadian. Electrochemical model parameter identification of a lithium-ion battery using particle swarm optimization method. *Journal of Power Sources*, 307:86–97, 2016.
- [13] Vinay Yadav Janga, Pranath Kumar Gourishetty, Biagio Carboni, Giuseppe Quaranta, and Walter Lacarbonara. Hysteretic tuned mass damper with bumpers for seismic protection: Modeling, identification, and shaking table tests. *Journal of Sound and Vibration*, 597:118816, 2025.
- [14] Jih-Lian Ha, Ying-Shieh Kung, Rong-Fong Fung, and Shao-Chien Hsien. A comparison of fitness functions for the identification of a piezoelectric hysteretic actuator based on the real-coded genetic algorithm. *Sensors and Actuators A: Physical*, 132(2):643–650, 2006.
- [15] Jingrong Wang, Qiao Peng, Jinhao Meng, Tianqi Liu, Jichang Peng, and Remus Teodorescu. A physics-informed neural network approach to parameter estimation of lithium-ion battery electrochemical model. *Journal of Power Sources*, 621:235271, 2024.
- [16] Rafael de O Teloli, Roberta Tittarelli, Maël Bigot, Lucas Coelho, Emmanuel Ramasso, Patrice Le Moal, and Morvan Ouisse. A physics-informed neural networks framework for model parameter identification of beam-like structures. *Mechanical Systems and Signal Processing*, 224:112189, 2025.
- [17] Hannes Stagge and Robert Güttel. The findability of microkinetic parameters by heterogeneous chemical reaction neural networks (hcrnns). *Chemical Engineering Journal*, 510:161460, 2025.
- [18] Benjamin C Koenig, Peng Zhao, and Sili Deng. Accommodating physical reaction schemes in dsc cathode thermal stability analysis using chemical reaction neural networks. *Journal of Power Sources*, 581:233443, 2023.
- [19] Patrick Kidger. On neural differential equations. *arXiv preprint arXiv:2202.02435*, 2022.
- [20] Ali Kashefi and Tapan Mukerji. Chatgpt for programming numerical methods. *Journal of Machine Learning for Modeling and Computing*, 4(2), 2023.
- [21] Zhehao Dong, Zhen Lu, and Yue Yang. Fine-tuning a large language model for automating computational fluid dynamics simulations. *Theoretical and Applied Mechanics Letters*, page 100594, 2025.
- [22] Sandeep Pandey, Ran Xu, Wenkang Wang, and Xu Chu. Openfoamgpt: A retrieval-augmented large language model (llm) agent for openfoam-based computational fluid dynamics. *Physics of Fluids*, 37(3), 2025.
- [23] Mohamed Elrefaie, Janet Qian, Raina Wu, Qian Chen, Angela Dai, and Faez Ahmed. Ai agents in engineering design: a multi-agent framework for aesthetic and aerodynamic car design. *arXiv preprint arXiv:2503.23315*, 2025.
- [24] Cyril Picard, Kristen M Edwards, Anna C Doris, Brandon Man, Giorgio Giannone, Md Ferdous Alam, and Faez Ahmed. From concept to manufacturing: Evaluating vision-language models for engineering design. *Artificial Intelligence Review*, 58(9):288, 2025.
- [25] Chandra Gnanasambandam, Martin Harrysson, Rikki Singh, and Aditi Chawla. How an ai-enabled software product development life cycle will fuel innovation. <https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/how-an-ai-enabled-software-product-development-life-cycle-will-fuel-innovation>, February 2025. McKinsey & Company. Accessed: 2025-09-07.

- [26] Matthias K Gobbert. Robertson’s example for stiff differential equations. *Arizona State University, Technical report*, 1996.
- [27] Weiqi Ji, Weilun Qiu, Zhiyu Shi, Shaowu Pan, and Sili Deng. Stiff-pinn: Physics-informed neural network for stiff chemical kinetics. *The Journal of Physical Chemistry A*, 125(36):8098–8106, 2021.
- [28] Ranjan Anantharaman, Yingbo Ma, Shashi Gowda, Chris Laughman, Viral Shah, Alan Edelman, and Chris Rackauckas. Accelerating simulation of stiff nonlinear systems using continuous-time echo state networks. *arXiv preprint arXiv:2010.04004*, 2020.
- [29] Kuang Yi Chen, Paul D Biernacki, A Lahrichi, and Alan Mickelson. Analysis of an experimental technique for determining van der pol parameters of a transistor oscillator. *IEEE transactions on microwave theory and techniques*, 46(7):914–922, 1998.
- [30] Yanli Wang, Xianghong Li, and Yongjun Shen. Study on mechanical vibration control of limit cycle oscillations in the van der pol oscillator by means of nonlinear energy sink. *Journal of Vibration Engineering & Technologies*, 12(1):811–819, 2024.
- [31] Jan Schöberl, Sebastian Ohneseit, Stefan Schaeffler, Dominic Förstermann, Linus Grahl, Andreas Jossen, Carlos Ziebert, and Markus Lienkamp. Thermal runaway characterization of cylindrical lithium-ion and sodium-ion batteries with various sizes and energy contents. *Journal of Power Sources*, 648:237240, 2025.
- [32] Luca Giuliano, Luigi Scrimieri, Simone Reitano, Davide Berti Polato, Alessandro Ferraris, Andrew Comerford, and Saakaar Bhatnagar. Experimental and simulation-based characterization of thermal runaway in lithium-ion batteries using altair simlab®. Technical report, SAE Technical Paper, 2025.
- [33] Chris Rackauckas. The numerical analysis of differentiable simulation: Automatic differentiation can be incorrect. <https://www.stochasticlifestyle.com/the-numerical-analysis-of-differentiable-simulation-automatic-differentiation-can-be-incorrect/>, April 2025. Stochastic Lifestyle blog post. Accessed 2025-09-07.
- [34] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.
- [35] Philipp Holl and Nils Thuerey. Φ_{flow} (PhiFlow): Differentiable simulations for pytorch, tensorflow and jax. In *International Conference on Machine Learning*. PMLR, 2024.
- [36] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN’95-international conference on neural networks*, volume 4, pages 1942–1948. iee, 1995.
- [37] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in neural information processing systems*, 31, 2018.
- [38] Weiqi Ji, Franz Richter, Michael J Gollner, and Sili Deng. Autonomous kinetic modeling of biomass pyrolysis using chemical reaction neural networks. *Combustion and Flame*, 240:111992, 2022.
- [39] Samuel Schoenholz and Ekin Dogus Cubuk. Jax md: a framework for differentiable physics. *Advances in Neural Information Processing Systems*, 33:11428–11441, 2020.

A Appendix

A.1 Particle Swarm Optimization (PSO)

Particle Swarm Optimization [36] is a zero-order optimization method that uses an initial distribution of particles in the search space and based on the ”fitness” of each particle computes new positions and velocities of particles in the search space. Eventually, the particles converge towards the optima.

The current categories of problems being solved in this paper take the general form

$$\begin{aligned} \min_{\mathbf{u}} f(\mathbf{u}), \\ \text{s.t} \end{aligned} \quad (15)$$

$$u_j^{min} \leq u_j \leq u_j^{max} \quad j = 1..M, \quad (16)$$

where f represents an objective function. $\mathbf{u}$ represents the input vector of design parameters (of length M), and each component of $\mathbf{u}$ has a u_j^{min} and u_j^{max} that they can take.

Once the objective function is set up, the i th particle positions ($\mathbf{u}$) and velocities are updated according to the equations:

$$\mathbf{u}^i = \mathbf{u}^i + \mathbf{v}^i, \quad (17)$$

$$\mathbf{v}^i = w\mathbf{v}^i + c_1r_1(\mathbf{u}_{best}^i - \mathbf{u}^i) + c_2r_2(\mathbf{u}_{best} - \mathbf{u}^i), \quad (18)$$

where c_1 , c_2 , r_1 , r_2 and w are constants.

A.2 Constrained Neural Ordinary Differential Equations (NODE)

Neural ordinary differential equations (Neural ODEs) [37] parameterize the vector field of a dynamical system and treat time as continuous. Given states $\mathbf{x}(t)$ and inputs $\mathbf{u}(t)$, the dynamics are modeled as

$$\dot{\mathbf{x}}(t) = f_{\theta}(\mathbf{x}(t), \mathbf{u}(t), t), \quad (19)$$

and numerical integration (e.g., Runge–Kutta or implicit schemes) yields trajectories for training and inference. This continuous-time formulation has been successfully applied across scientific domains where data are irregularly sampled and dynamics are naturally specified by differential equations. Since gradients are obtained by differentiating through the numerical solver (adjoint or forward/Reverse-mode AD), updates reflect the true sensitivity of simulated trajectories to parameters and network weights under the chosen discretization, improving stability relative to finite-difference estimates. The parameters being search for can then be updated using first order methods such as Adam or methods like L-BFGS. In this work, L-BFGS with line search is used during gradient updates to fine tune the solution.

For parameter estimation, the ODE system being estimated can be treated as a special case of a constrained neural ordinary differential equation, such that the network architecture is constrained by the ODE structure. Several works have utilized this method, such as Chemical Reaction Neural Network (CRNN) for parameter inference [10, 18, 38]. This method is usually used in conjunction with heuristic methods or population methods, since it needs a good initial condition to begin its search so that the gradients are well defined at the start. In this work, the differentiable simulation pipeline is written in JAX [39], using the Diffrax [19] library.

A.3 Examples of Agentic Corrections

This section demonstrates the benefits of the agentic workflow beyond just simplified and streamlined problem orchestration. A couple of examples are discussed where the agent identifies semantic and implementational errors in user inputs, automatically corrects them, reports the corrections and returns the corrected files.

The correction step itself is divided into two parts, report generation and error correction

A.3.1 Robertson’s Equations

The agent is given the user input files corresponding to the Robertson system, and asked to generate a report on their correctness. The agent identifies two semantic errors, automatically rewrites the files with corrections, and then re-checks the files. Once it finds no critical errors, it ends step II of the workflow, allowing the user to proceed to step III. Figure 9 shows the workflow of agentic correction of input files based on suboptimal inputs. This correction loop runs for only 1 iteration, however it can be configured to be run for more iterations if necessary.

User inputs with incorrect or suboptimal settings or features (highlighted)

```
<?xml version="1.0" ?>
<FIT>
  <EXPERIMENT>
    <FILENAME>
      <E> FILENAME_DATA = robertson_data.csv </E>
    </FILENAME>
  </EXPERIMENT>

  <MODEL>
    <TRAINABLE_PARAMETERS>
      <P> N_TRAINABLE_PARAMETERS = 3 </P>
    </TRAINABLE_PARAMETERS>
    <TRAINABLE_PARAMETER_DESCRIPTION>
      <PARAM>
        <P> PARAMETER_NAME= k1 </P>
        <P> MIN_VAL=0.001 </P>
        <P> MAX_VAL=10.0 </P>
        <P> LOGSCALE= Y </P>
      </PARAM>
      <PARAM>
        <P> PARAMETER_NAME= k2 </P>
        <P> MIN_VAL=5E3 </P>
        <P> MAX_VAL=5E10 </P>
        <P> LOGSCALE= N </P>
      </PARAM>
      <PARAM>
        <P> PARAMETER_NAME= k3 </P>
        <P> MIN_VAL=1E0 </P>
        <P> MAX_VAL=1E8 </P>
        <P> LOGSCALE= Y </P>
      </PARAM>
    </TRAINABLE_PARAMETER_DESCRIPTION>
  </FIXED_PARAMETER_DESCRIPTION>
</FIT>
```

```
def user_defined_system(t, y,
trainable_parameters, fixed_parameters,
dataset, t_eval):
    .
    .

def _compute_loss_problem(solution_time,
solution, dataset, trainable_parameters,
fixed_parameters):

    return
    np.sqrt(np.mean(np.square(solution -
dataset)))

def writeout_description(solution_time,
solution, dataset, trainable_parameters,
fixed_parameters):
    .
    .
```

Agent generates an initial report of issues

Critical Errors

None detected

Warnings

1. Parameter k2 Should Be Searched in Logspace:
 - In the XML, k2 has a MIN_VAL = 5E3 and MAX_VAL = 5E10, which span 7 orders of magnitude. It is currently set to LOGSCALE= N. It is strongly recommended to change LOGSCALE to Y for k2 to facilitate efficient and meaningful parameter search.
2. Loss Function Output May Not Be Properly Scaled:
 - In _compute_loss_problem, the loss is calculated as the root mean squared error between solution and dataset: `return np.sqrt(np.mean(np.square(solution - dataset)))`. There is commented-out code that would scale this by the maximum of the dataset, which would help ensure the loss is between 0 and 1. Consider uncommenting and using the scaling logic to avoid optimization stability problems.

Number of critical errors: 0

Agent makes changes and summarizes them to user

Changes summary

Summary of changes:

- In the XML: For parameter k2, changed '`<P> LOGSCALE= N </P>`' to '`<P> LOGSCALE= Y </P>`' to ensure search is performed in logspace.
- In the .py file: Uncommented and now use scaling by the maximum of the dataset in '`_compute_loss_problem`' to keep loss values scaled and avoid optimization instability. The previous (unscaled) loss is now commented out. No other changes made.

#####



Figure 9: Workflow of correction of suboptimal inputs in the Robertson parameter estimation problem, in step II of the overall workflow. The agent examines the input files and corrects errors it considers significant, and edits the input files while apprising the user of the changes made. It then again reads the input files, and saves a final report.

A.3.2 Battery Thermal Runaway Modeling

The agent is given the user input files corresponding to the thermal runaway system, and asked to generate a report on their correctness. The agent identifies missing information in the input XML and an incorrectly named function. The agent corrects them before re-checking the inputs, and given it sees no more critical errors, ends step II. The user may choose to address the generated warnings. Figure 10 shows the workflow of agentic correction of input files based on suboptimal inputs. This correction loop runs for only 1 iteration, however it can be configured to be run for more iterations if necessary.

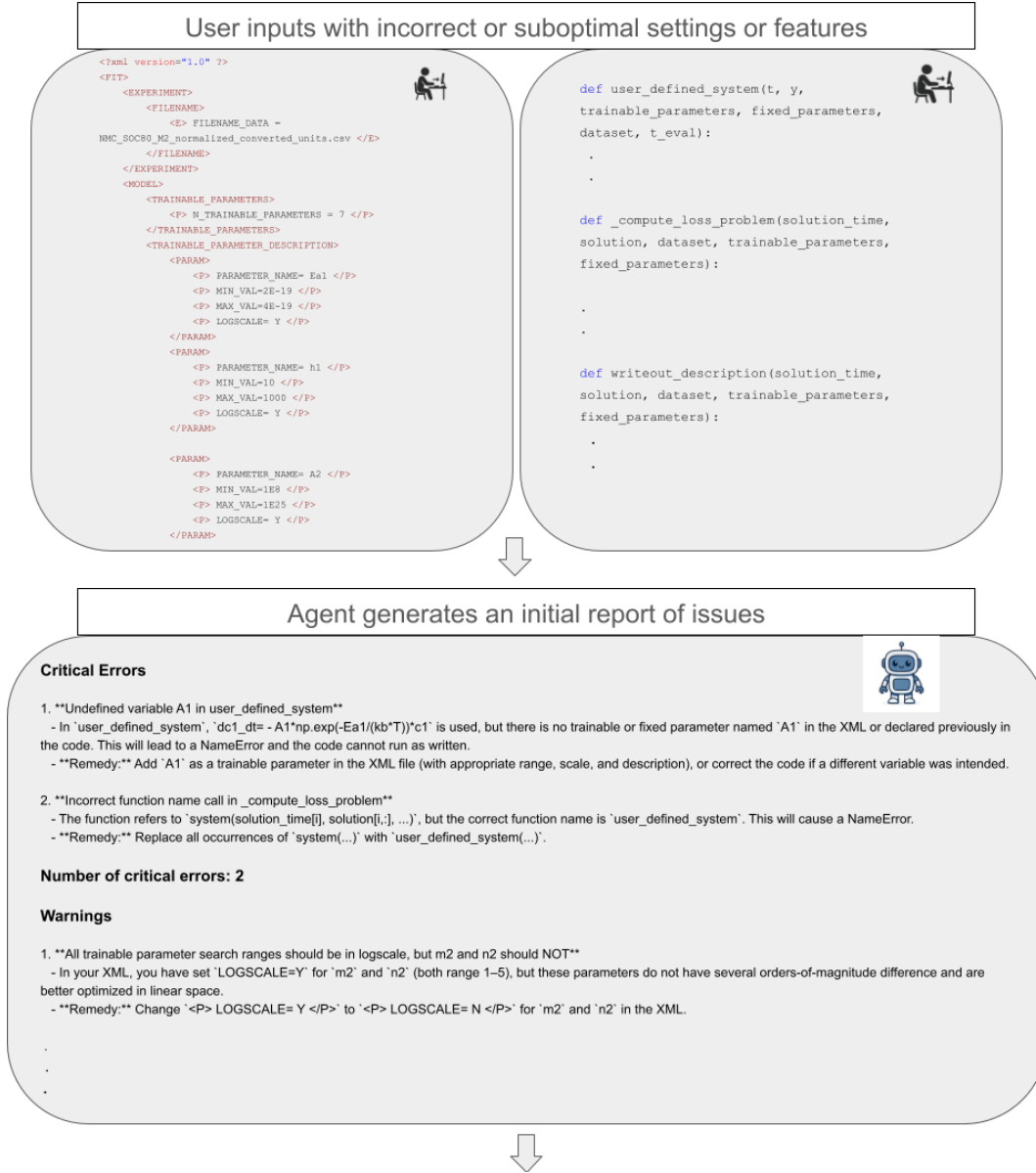




Figure 10: Workflow of correction of suboptimal inputs in the battery thermal runaway model parameter estimation problem, in step II of the overall workflow.